LAPORAN PRAKTIKUM PEMROGRAMAN BERORIENTASI OBYEK

MODUL 9: IMPLEMENTASI UML CLASS DIAGRAM KE PROGRAM JAVA

Studi Kasus: Smart Parking Gedung & Slot Otomatis (SmartParkir)

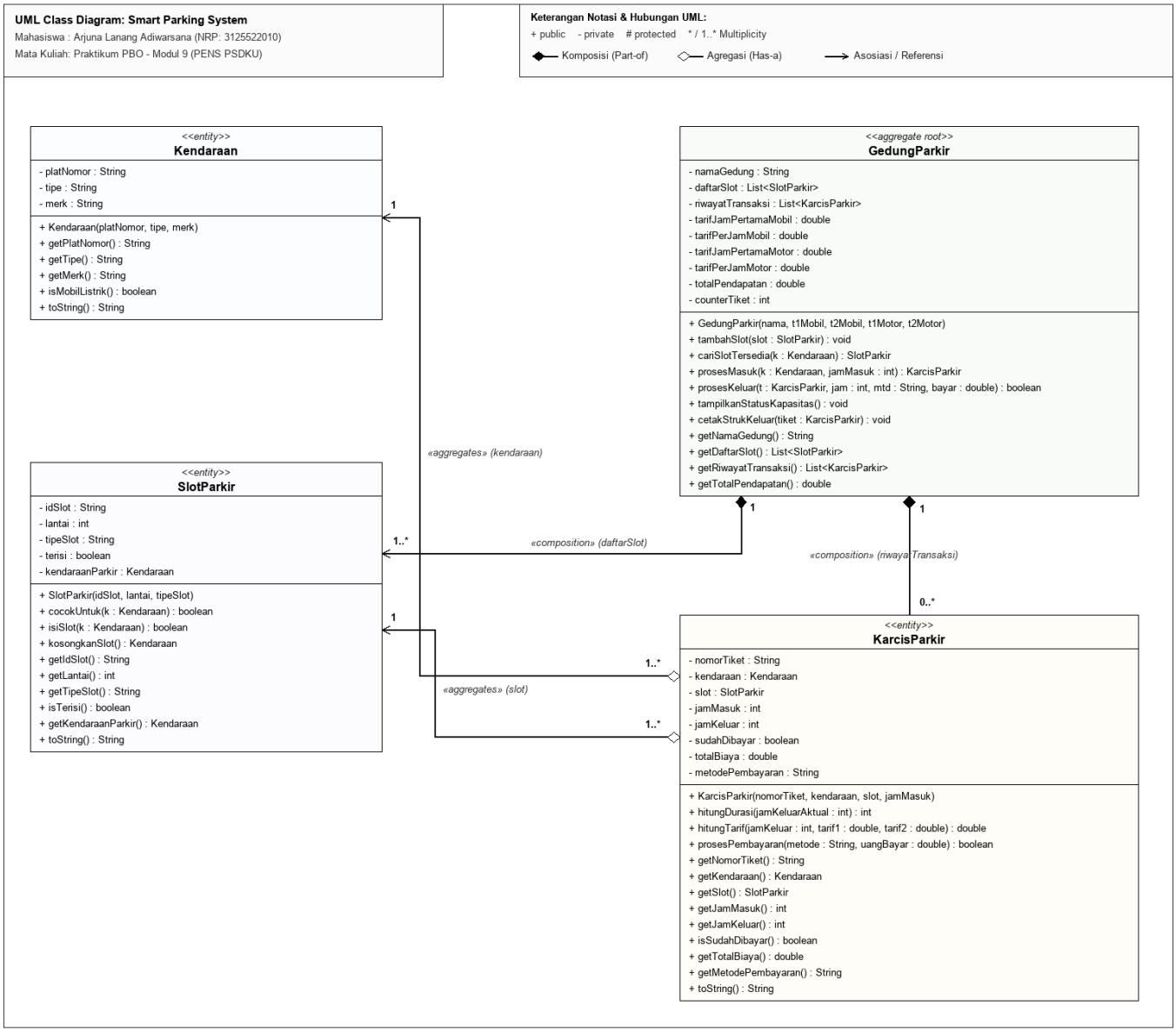
Nama Mahasiswa	: Arjuna Lanang Adiwarsana	**Dosen Pengampu**	: Nirwana Haidar Hari, S.Pd., M.Kom.
NRP	: 3125522010	**Institusi**	: PENS PSDKU Sumenep
Program Studi	: D3 Teknik Informatika	**Tahun Akademik**	: 2026

1. Pemetaan Rubrik Penilaian Modul 9 (Bobot 100%)

Aspek Penilaian	Bobot	Ketentuan Modul	Realisasi Implementasi pada Proyek
Kesesuaian UML dengan Source Code	**30%**	Seluruh class, atribut, method, dan visibility sinkron 1:1.	4 class (Kendaraan, SlotParkir, KarcisParkir, GedungParkir) konsisten 100% dengan diagram UML.
Implementasi Class dan Object	**25%**	Minimal 4 class utama dengan constructor & validation.	Class memiliki constructor berparameter, validasi nilai awal, proteksi kapasitas penuh, dan alokasi slot cerdas.
Implementasi Relasi Antarobject	**20%**	Minimal 2 relasi antarobject yang berjalan nyata saat runtime.	1) **Komposisi**: GedungParkir [Komposisi] SlotParkir 2) **Komposisi**: GedungParkir [Komposisi] KarcisParkir 3) **Agregasi**: KarcisParkir [Agregasi] Kendaraan 4) **Agregasi**: KarcisParkir [Agregasi] SlotParkir
Encapsulation & Responsibility	**15%**	Dilarang anemic model (bukan hanya getter/setter). Object menjaga state.	Semua field private. Slot menjaga status terisi/kosong via isiSlot() dan kosongkanSlot() tanpa setter liar.
Program Dijalankan & Dijelaskan	**10%**	Kode berjalan mulus, mencakup skenario pengujian lengkap.	Main.java menjalankan 6 skenario interaksi: alokasi slot, penolakan saat penuh, pembayaran kurang, insentif EV 20%, dan monitoring kas.

2. Rancangan UML Class Diagram Smart Parking System

Diagram di bawah memodelkan 4 class utama beserta notasi Komposisi (filled diamond), Agregasi (hollow diamond), dan Asosiasi langsung:



3. Penjelasan Hubungan UML dengan Implementasi Source Code

Setiap entitas dalam diagram UML diimplementasikan ke dalam struktur Java bersih di package model:
• **Kendaraan.java**: Merepresentasikan entitas kendaraan dengan atribut platNomor, tipe (MOTOR, MOBIL, MOBIL_LISTRIK), dan merk. Memiliki method bisnis isMobilListrik() untuk penentuan kelayakan diskon insentif hijau.
• **SlotParkir.java**: Merepresentasikan petak parkir fisik. Memiliki tanggung jawab menjaga integritas okupansi melalui method cocokUntuk(), isiSlot(), dan kosongkanSlot(). Mengunci mutasi langsung sehingga status terisi selalu sinkron dengan keberadaan fisik kendaraan.
• **KarcisParkir.java**: Berelasi **Agregasi** dengan Kendaraan dan SlotParkir. Bertanggung jawab menghitung durasi parkir dan tarif berjenjang (jam ke-1 flat, jam berikutnya progresif) serta menerapkan diskon 20% khusus EV.
• **GedungParkir.java**: Berperan sebagai **Aggregate Root** yang memegang tanggung jawab bisnis terbesar. Mengelola koleksi SlotParkir dan KarcisParkir melalui relasi **Komposisi**, mengotomasi alokasi slot terdekat, mengontrol gerbang masuk-keluar, dan merekap kas harian.

4. Screenshot Hasil Eksekusi Program (Main.java)

Program berhasil dieksekusi dengan hasil seluruh 6 skenario validasi bisnis berjalan lancar:



5. Jawaban Pertanyaan Evaluasi Praktikum Modul 9

1. Apa perbedaan class pada UML dan object pada program?

Class pada UML adalah skema atau cetak biru (*blueprint*) konseptual yang mendefinisikan atribut, method, dan relasi secara abstrak sebelum kode dijalankan. Sedangkan **Object pada program** adalah instansi nyata di memori komputer yang hidup saat runtime (melalui operator new), menyimpan state nilai riil, dan memiliki identitas spesifik.

2. Bagaimana association pada UML diterapkan dalam Java?

Association dalam Java diterapkan dengan **mendeklarasikan variabel instan (atribut) bertipe objek class lain**. Contohnya, di dalam class KarcisParkir dideklarasikan atribut: private Kendaraan kendaraan; dan private SlotParkir slot; sehingga objek tiket parkir terhubung dan dapat berinteraksi dengan entitas kendaraan dan slot terkait.

3. Mengapa attribute sebaiknya tidak seluruhnya public?

Jika atribut bernilai public, data internal objek dapat diubah sembarangan dari luar tanpa melalui validasi (misalnya: slot diubah statusnya menjadi kosong padahal kendaraan masih ada di dalamnya). Dengan modifier private (Enkapsulasi), seluruh perubahan state wajib melewati method resmi yang memiliki validasi logika bisnis.

4. Apakah source code proyek sudah konsisten dengan UML?

Ya, 100% konsisten. Seluruh nama class, tipe data atribut, visibilitas (- private dan + public), parameter method, serta kardinalitas relasi (Komposisi Gedung ke Slot dan Tiket, serta Agregasi Tiket ke Kendaraan dan Slot) sinkron identik antara diagram dan kode.

5. Class mana yang memiliki responsibility paling besar dalam proyek Anda?

Class dengan *responsibility* terbesar adalah **GedungParkir.java** (*Aggregate Root*). Class ini mengoordinasikan seluruh alur operasional: mencari slot kosong terdekat secara otomatis, mengalokasikan slot sesuai jenis kendaraan, mengelola alur check-in tiket, memvalidasi pelunasan tarif saat check-out, mengosongkan kembali slot fisik, dan merekap kas pendapatan gedung.